DEPARTMENT OF EXAMINATIONS

JAVA INSTITUTE FOR ADVANCED TECHNOLOGY

Unit Name: Object Oriented Design Patterns II | Unit ID: JIAT/OODP II
Assessment Identifier: JIAT/OODP II /EX/01

SMARTCITY360

MUNICIPAL DIGITAL OPERATIONS PLATFORM

Design Pattern Implementation and Architectural Assessment Report
Scenario: UrbanTech Solutions | Target City: Metropolis Neo-Prime

Candidate Name: Dhanushka Maduranga
Student Registration No: [Student Registration Number]
Academic Specialization: Software Engineering
Submission Date: November 2026

Declaration: I certify that this report, the accompanying source code, test suite and demonstration materials are my own individual work, prepared in accordance with the Java Institute Academic Regulations. All sources are acknowledged in the reference list.

Contents

1. Introduction and Scenario Overview
2. Part A: Dynamic Security Wrappers for Data Streams (Decorator)
3. Part B: Hierarchical Structure of Smart Zones (Composite)
4. Part C: Escalation of Citizen Complaints (Chain of Responsibility)
5. Part D: Efficient Management of Smart Street Lights (Flyweight)
6. Part E: Interpreting City Policies Defined in DSL (Interpreter)
7. Part F: Final Project Demonstration, VIVA and Documentation
8. Conclusion and References
Appendix A: Source Code Screenshot Catalog

1. Introduction and Scenario Overview

SmartCity360 is a digital platform that manages infrastructure, devices, services and resident interactions for the city of Metropolis Neo-Prime. As the senior developer at UrbanTech Solutions, I designed and implemented five core components. Each component solves one problem with exactly one Gang of Four (GoF) design pattern, as the assessment requires. Procedural code and rigid inheritance would have produced tight coupling, memory bloat and unmaintainable conditional logic, so each pattern was chosen to address one specific problem.

Part	Scenario	Pattern	Core problem solved
A	Security layers on energy-meter streams	Decorator	Add or remove compression, encryption and authentication without changing core logic
B	City → district → street → building zones	Composite	Apply one command uniformly at any level of the hierarchy
C	Complaint escalation by severity	Chain of Responsibility	Route requests through staff, supervisors and managers without coupling the sender to them
D	25,000 street lights	Flyweight	Share identical hardware data to minimise memory
E	City policies written in a mini-language	Interpreter	Parse and evaluate rules at runtime without recompiling

Each of Parts A to E below answers the three assessment questions in order: (1) identify and justify the pattern, (2) explain its mechanism, and (3) evaluate it. The implementation is a Java 17 servlet backend with MySQL persistence. Source files are under `com.smartcity.pattern.*`, and 35 automated tests verify the behaviour (Section 7.3).

2. Part A: Dynamic Security Wrappers for Data Streams

> *Energy meters continuously stream readings across public and private networks. The platform must add layers such as compression, encryption and authentication to these streams, and the layers must be addable without altering the core logic.*

A.1 Identification and Justification (3 marks)

Selected pattern: Decorator (structural; Gamma et al., 1994). A decorator wraps an object, implements the same interface, and adds behaviour before or after delegating to the wrapped object. The justification rests on three points.

- **Avoiding class explosion.** With three optional features, static inheritance needs $2^3 = 8$ subclasses (for example `CompressedEncryptedAuthenticatedStream`). A fourth feature, such as anonymisation, needs 16, and a fifth needs 32. The Decorator pattern needs only N small classes (`CompressionDecorator`, `EncryptionDecorator`, `AuthenticationDecorator`) plus one interface and one abstract base. All 2^N combinations are still available, but they are assembled at runtime rather than written as classes.
- **Open/Closed Principle.** The core class `EnergyStreamData` is never modified. New layers are new classes (Martin, 2008).
- **Runtime flexibility.** Inheritance fixes behaviour at compile time. With decorators, operators can wrap or unwrap layers per link, for example enabling compression on metered cellular backhaul or encryption on an untrusted WAN.

Why not the alternatives? Strategy swaps one algorithm for another but cannot stack several transformations in sequence. A Proxy controls access to one object but is not designed to compose repeatedly. A configuration-flag approach (`if (encrypt) …`) puts every feature into one class and breaks the Open/Closed Principle. Only Decorator gives stackable, independently testable layers behind an unchanged interface.

A.2 How the Pattern Supports Flexible, Layered Enhancement (4 marks)

The `StreamData` interface (`getData`, `getPayloadBytes`, `getPayloadSize`, `getDescription`) is implemented by both the concrete component `EnergyStreamData` and the abstract `StreamDataDecorator`. The decorator holds a `StreamData` reference (`wrappee`) and delegates to it by default. Because a decorator is itself a `StreamData`, it can wrap the core object or another decorator, so layers nest to any depth:

```
StreamData s = new EnergyStreamData("STR-101", 230.5, 14.2, 3.27);
s = new CompressionDecorator(s);          // 1. GZIP
s = new EncryptionDecorator(s, aesKey);   // 2. AES-256-GCM
s = new AuthenticationDecorator(s, hmacKey); // 3. HMAC-SHA256
```

Each layer transforms the output of the layer beneath it, so the enhancements stack without the client knowing how many exist. Three design decisions make the stack correct:

- **Compression before encryption.** AES-GCM output looks like random noise, which GZIP cannot compress. Compressing the plain JSON first is the only order that saves bandwidth.
- **Authenticated encryption.** `EncryptionDecorator` uses AES-256-GCM with a fresh 12-byte random IV and a 128-bit tag, so tampering with the ciphertext or IV raises `AEADBadTagException` on decryption (Dworkin, 2007).

- **Outer HMAC with a separate key.** `AuthenticationDecorator` computes HMAC-SHA256 over the whole JSON envelope (cipher parameters, IV and ciphertext). Using a separate key lets perimeter gateways verify and drop forged packets without ever holding the AES decryption key. GCM already detects tampering, so the outer HMAC is an operational choice, not a cryptographic necessity. HMAC proves integrity and shared-secret origin but not non-repudiation (Krawczyk et al., 1997).

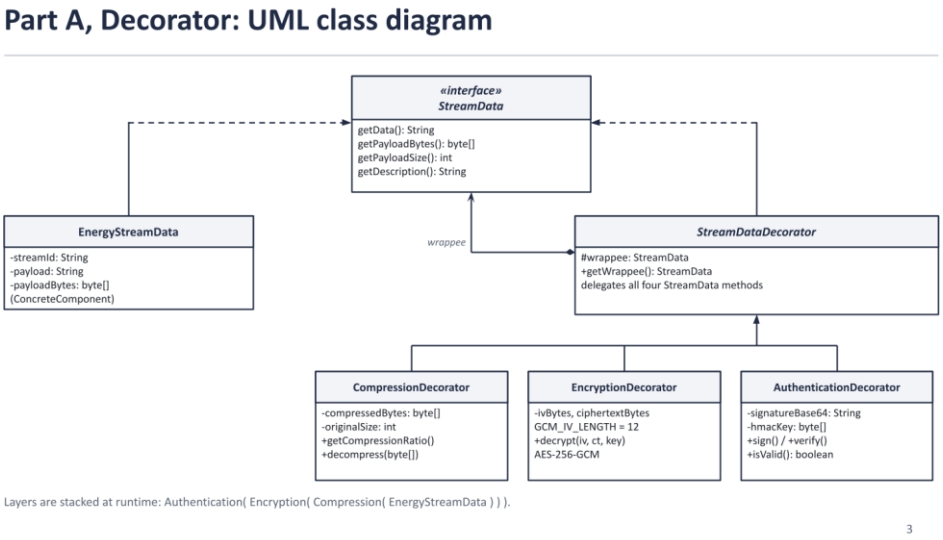


Figure 2.1: UML class diagram, Decorator pattern (Part A)

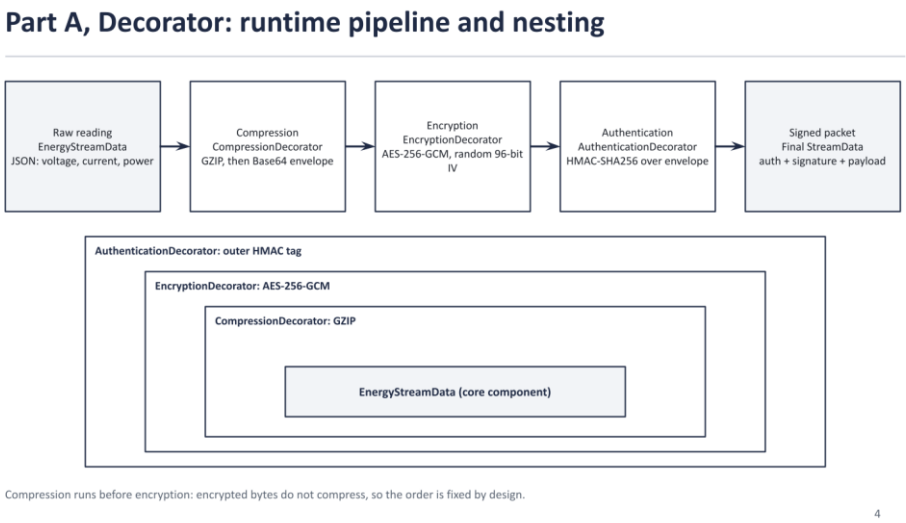


Figure 2.2: Stream wrapping pipeline

A.3 Long-Term Benefits: Reusability and Extensibility (8 marks)

Reusability. Each concern lives in one class and no decorator depends on another at compile time. Different sensors reuse the same classes in different stacks: a substation uses compression, encryption and authentication; a low-power air-quality sensor on a private network uses compression only; a public analytics feed uses authentication only. No logic is duplicated across ingestion handlers, which removes copy-paste code and keeps each method short.

Extensibility. A new requirement becomes one new class that extends `StreamDataDecorator`. If regulators mandate post-quantum signatures, only a new decorator is written. The polling engine,

models and database code stay untouched, so there is no regression risk in existing layers. Client controllers depend only on `StreamData`, so the transformation algorithms can change freely behind it.

Measured behaviour. My tests show the benefit is real but must be reported honestly. A single 72-byte JSON reading grows to 92 bytes under GZIP because of the 10-byte header and 8-byte trailer, and the full pipeline turns it into a 468-byte envelope (IV, GCM tag, Base64 expansion and a 64-character HMAC). On batched telemetry of 1 KB to 64 KB the repeated JSON keys let GZIP achieve roughly 65% to 75% reduction. The pipeline should therefore compress batches, not individual packets.

Cost of change over time. Because behaviour is isolated, each decorator can be unit-tested alone and then in combination. Replacing GZIP with another codec, or rotating cryptographic keys, touches one class. Under the inheritance alternative every combination class that embeds the old logic would need editing. This is where the pattern pays back most over a multi-year platform life.

Drawbacks. Decorators create many small objects, and the correct order (compress, encrypt, authenticate) is a convention that nothing in the type system enforces. A mis-ordered stack still runs but silently loses compression. Debugging means stepping through several wrappers, and `instanceof` checks on the outermost object no longer reveal the core type. I mitigate this by assembling stacks in one place and by testing round trips (tests 01 to 06 cover the pipeline, decompression, decryption, tampering, corrupted payloads and invalid keys).

3. Part B: Hierarchical Structure of Smart Zones

> *City zones (city, district, street, building) must be represented as a hierarchy so that commands such as "shutdown power" or "report usage" can be applied uniformly at any level.*

B.1 Identification and Justification (3 marks)

Selected pattern: Composite (structural; Gamma et al., 1994). The zones form a classic part-whole tree: a city contains districts, districts contain streets, and streets contain buildings. Composite lets client code treat a single building and an entire city through one interface, `ZoneComponent`.

- **Uniform treatment.** `BuildingLeaf` and `ZoneComposite` share one contract, so the caller does not need to know which level it holds.
- **No type checking.** Without Composite, controllers need `if (zone instanceof City) … else if (zone instanceof District) …` chains that break whenever a level is added. With Composite the client writes one line and polymorphism handles the traversal: `zone.executeCommand(ZoneCommand.SHUTDOWN_POWER);`

B.2 UML Class Diagram and Mechanism (4 marks)

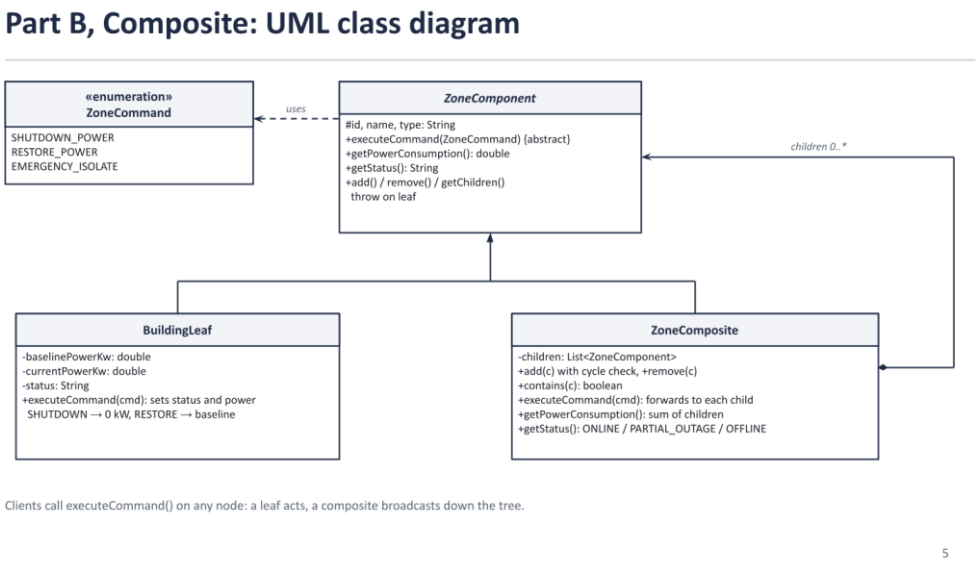


Figure 3.1: UML class diagram, Composite pattern (Part B)

The roles are: **Component** `ZoneComponent` (declares `executeCommand`, `getPowerConsumption`, `getStatus`, and `add`/`remove`/`getChildren`); **Leaf** `BuildingLeaf` (holds relay status and power draw); **Composite** `ZoneComposite` (holds a `List<ZoneComponent>` and delegates). Commands are a type-safe enum, `ZoneCommand` (`SHUTDOWN_POWER`, `RESTORE_POWER`, `EMERGENCY_ISOLATE`, `REPORT_USAGE`), so a misspelt string cannot fail silently. A leaf handles the command itself (shutdown sets status OFFLINE and draw to 0 kW; restore returns it to its rated baseline), while a composite forwards it to every child:

```
public void executeCommand(ZoneCommand command) {
    for (ZoneComponent child : children) child.executeCommand(command);
}
```

B.3 Usefulness for a Large and Growing City (8 marks)

Uniform command cascade. A shutdown sent to the City composite passes to each District, then each Street, then each BuildingLeaf. The traversal is a depth-first recursion costing O(M) for M nodes in the subtree, and the caller needs no knowledge of the depth. The same call works on one building or the whole city.

Growth without change. New districts, industrial corridors or solar micro-grids are inserted anywhere in the tree with no change to client code. A new kind of container, such as a `MetroTransitComposite`, only has to extend `ZoneComponent`. This follows the Open/Closed Principle and suits a city that keeps expanding.

Recursive aggregation. `getPowerConsumption()` returns a building's own draw for a leaf and the sum of its children for a composite, so dashboards ask one question at any level, and operations teams get live roll-ups for a street, district or city. `getTotalPowerConsumption()` is a convenience alias.

Safety. Calling `add()` on a leaf throws `UnsupportedOperationException`, and `ZoneComposite` rejects null children, self-addition and circular references (tests 13 and 27). Invalid command strings raise a clear error listing the supported commands (test 12).

Operational scenario. During a grid emergency, an operator can isolate one district with `EMERGENCY_ISOLATE`, query its remaining load with `REPORT_USAGE`, and later restore it with `RESTORE_POWER`, all through the same interface. A restore returns each building to its own rated baseline, not to a single default value, so the city returns to its previous profile (tests 09 and 10).

Trade-offs. Putting child-management methods on the common interface favours transparency over safety, because leaves expose methods they cannot support; I accept that and guard it with runtime exceptions. The design is very general, so a type-specific rule (for example, "only streets can be isolated") needs extra checks. Very deep trees recurse deeply and are held in memory, and a broadcast has no built-in rollback if one leaf fails. For the expected four-level hierarchy these costs are small compared with the removed traversal code.

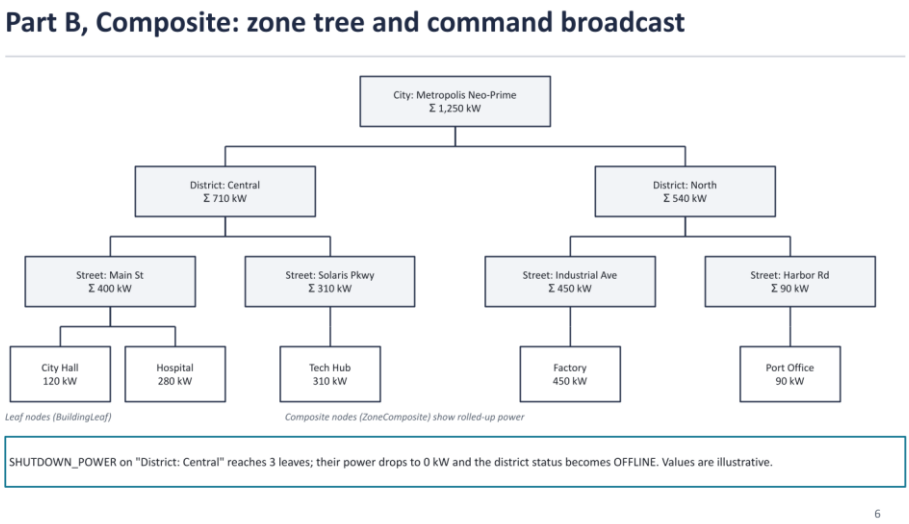


Figure 3.2: Zone tree and command broadcast

4. Part C: Escalation of Citizen Complaints

> *When a resident submits a complaint, the system must escalate it from local staff to supervisors to city managers depending on severity, while keeping a reliable record of what happened.*

C.1 Identification and Justification (3 marks)

Selected pattern: Chain of Responsibility (behavioral; Gamma et al., 1994). A request travels along a linked chain of handlers until one of them takes responsibility for it.

- **Sender and receivers are decoupled.** The resident or servlet submits to the head of the chain and never learns which official handles the case.
- **The escalation path is configurable.** Handlers are linked at runtime with `setNext()`, so a path can be reordered, shortened or extended. For example, an emergency handler could be prepended during severe weather without editing existing handlers.
- **Single Responsibility.** Each level of authority is one class that holds only its own thresholds and routing decisions.

C.2 How the Pattern Enables Decoupled Request Handling (4 marks)

The abstract `ComplaintHandler` stores a `nextHandler`, offers `setNext()` (which returns the next handler so calls can be chained), and declares `handle(Complaint)`. Each concrete handler either processes the complaint or forwards it. `ComplaintChainFactory` wires the production chain:

```
ValidationTriage -> LocalStaff -> Supervisor -> CityManager -> TerminalFallback
```

Handler	Takes	Outcome
ValidationTriageHandler	Missing or invalid severity	Role TRIAGE_DISPATCH_OFFICER, status VALIDATION_REQUIRED; never defaults to LOW
LocalStaffHandler	LOW	Status ASSIGNED (routine maintenance)
SupervisorHandler	MEDIUM	Status ESCALATED_SUPERVISOR (e.g. water leaks, signal faults)
CityManagerHandler	HIGH, CRITICAL	Status EXECUTIVE_ESCALATION; CRITICAL gets crisis mobilisation notes
TerminalFallbackHandler	Anything unresolved	Role FALLBACK_DISPATCH_DESK, status FALLBACK_ASSIGNED, with audit record

Worked example. A CRITICAL gas-leak report enters at `ValidationTriageHandler`, which finds a valid severity and forwards it. `LocalStaffHandler` and `SupervisorHandler` do not accept CRITICAL and forward it in turn, and `CityManagerHandler` handles it with status EXECUTIVE_ESCALATION. Each hop appends an audit record, so the final result shows the complete route. At no point does the servlet know which handler acted.

The triage handler fixes a hazard I identified in legacy systems: a missing severity silently became LOW, so an unclassified gas leak could reach a routine street crew. Here an unknown severity goes to a human triage officer instead. The client only holds the head of the chain, so handlers can change without affecting it.

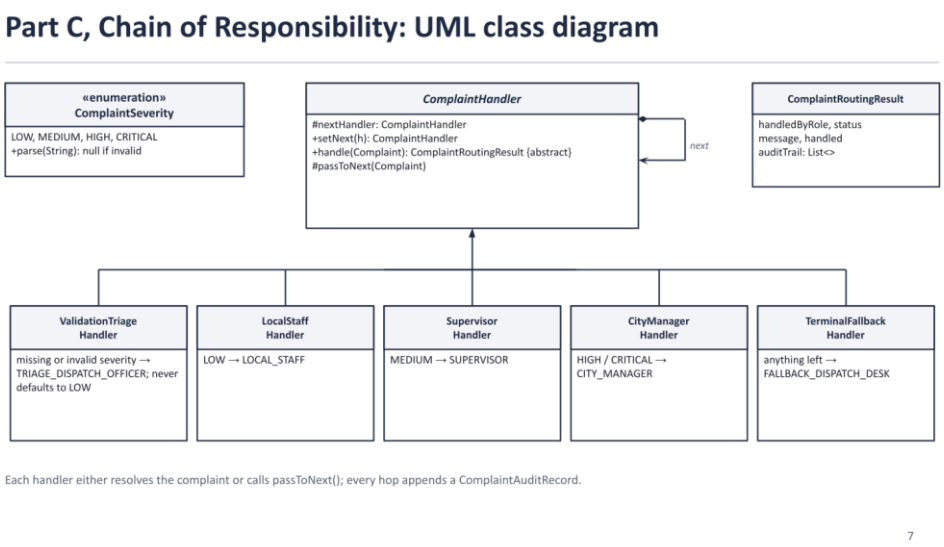


Figure 4.1: UML class diagram, Chain of Responsibility (Part C)

C.3 Benefits and Drawbacks: Flexibility and Traceability (8 marks)

Flexibility (benefits). Different departments can have different chains (for example Police or Public Works) built by separate factory methods without touching controllers. The pattern removes the large `switch` on severity that would otherwise violate the Open/Closed Principle, and new tiers are added by inserting one handler.

Traceability (benefits). `ComplaintRoutingResult` accumulates an immutable, chronological list of `ComplaintAuditRecord` entries, each holding the handler name and role, an ISO-8601 timestamp, the action and the reason. The route (for example Triage → LocalStaff → Supervisor → CityManager) can therefore be reconstructed for any ticket, which supports administrative accountability (test 21 verifies order and immutability).

Drawbacks.

- **Latency grows with chain length.** Each node runs a check and allocates an audit record; with five nodes this is negligible, but it is slower than a direct indexed dispatch.
- **Harder debugging.** Routing is distributed across objects, so a stack trace spans several delegation frames, and finding why a ticket went to a given tier means inspecting the chain wiring.
- **Requests can fall off the end.** A badly wired chain may leave a complaint unhandled. I mitigate this with `TerminalFallbackHandler`. In the standard chain it is unreachable because triage plus the three tiers cover every valid severity; it exists as a fail-safe for reconfigured chains, such as a department chain without an executive tier (test 20).
- **Order dependence and audit overhead.** Correctness relies on wiring discipline, and the per-ticket audit objects add garbage during complaint surges.

Overall, the flexibility and the audit trail outweigh these costs for a municipal service where accountability matters more than microseconds.

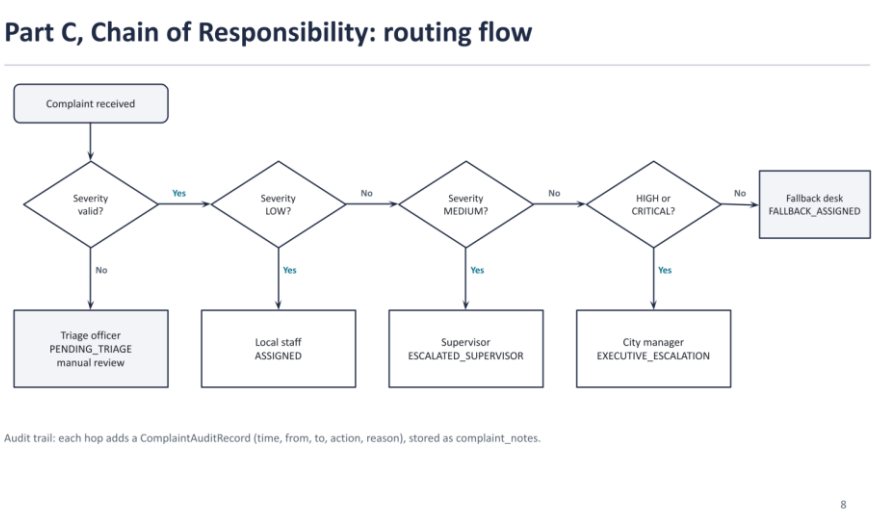


Figure 4.2: Complaint routing flow

5. Part D: Efficient Management of Smart Street Lights

> *The city operates 25,000 smart streetlights with identical behaviour and configuration but different locations. The system must minimise memory use.*

D.1 Identification and Justification (3 marks)

Selected pattern: Flyweight (structural; Gamma et al., 1994). Thousands of poles share the same model, lamp type, wattage, lumen rating, colour temperature and firmware. Creating 25,000 fully independent objects stores the same data thousands of times, which causes heap bloat, more garbage collection and, at larger scale, out-of-memory risk. Flyweight stores the shared data once and lets every pole reference it. It also separates immutable hardware data from the mutable, location-dependent state of each pole.

D.2 Handling Shared (Intrinsic) and Unique (Extrinsic) Properties (4 marks)

- **Intrinsic state (shared, immutable):** model, lampType, wattage, maxLumens, colorTemperatureK, firmwareVersion. It is stored once in `SharedStreetlightProfile`, created and cached by `StreetlightProfileFactory`.
- **Extrinsic state (unique per pole):** poleId, latitude, longitude, brightnessLevel and operational status. It is stored in a lightweight `StreetlightContext`, which also holds a reference to its shared profile.

The factory keys the cache on the combined hardware attributes and uses `ConcurrentHashMap.computeIfAbsent()`, so concurrent ingestion threads never create duplicate profiles and no global lock is needed (Goetz et al., 2006):

```
String key = model+"|"+lampType+"|"+wattage+"|"+maxLumens+"|"+colorTemp+"|"+firmware;
return cache.computeIfAbsent(key, k -> new SharedStreetlightProfile(...));
```

Tests 22 to 25 confirm that identical specifications return the same instance, different specifications return distinct instances, eight concurrent threads still produce one profile, and two poles keep independent brightness while sharing a profile.

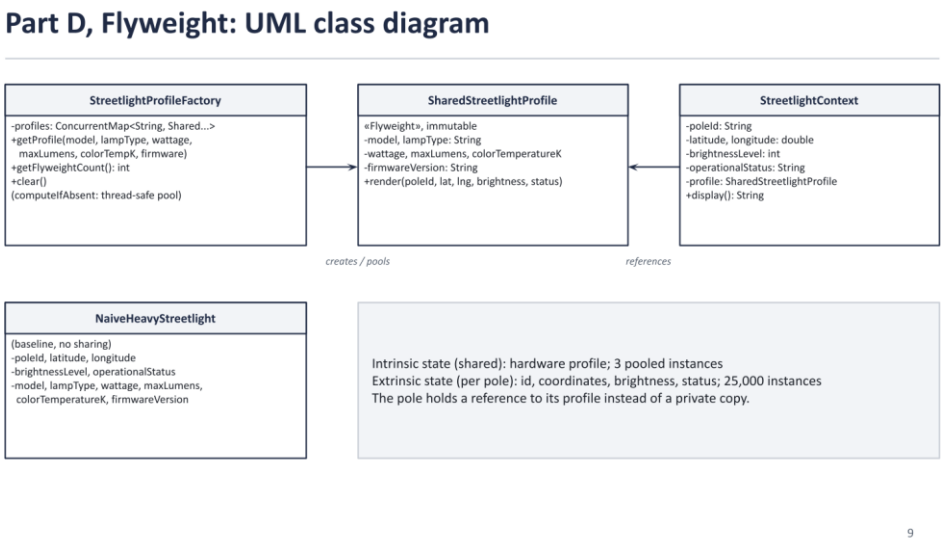


Figure 5.1: UML class diagram, Flyweight pattern (Part D)

D.3 Performance Impact at Tens of Thousands of Objects (8 marks)

Analytical model. On 64-bit HotSpot with compressed OOPs, I estimate a naive object (header, primitives, five string references and duplicated strings) at 288 bytes. A flyweight context is 80 bytes, and each shared profile is 208 bytes. These figures are the constants in `FlyweightBenchmark.java`:

- Naive: 25,000 × 288 B = 7,200,000 B (6.87 MiB).
- Flyweight: 25,000 × 80 B + 3 profiles × 208 B = 2,000,624 B (1.91 MiB).
- Reduction: (7,200,000 − 2,000,624) / 7,200,000 = **72.2%**.

Scaling behaviour. The benchmark was run at four fleet sizes (OpenJDK 17):

Poles	Naive (layout)	Flyweight (layout)	Reduction	Naive time	Flyweight time
1,000	0.27 MB	0.08 MB	71.99%	11 ms	12 ms
10,000	2.75 MB	0.76 MB	72.20%	16 ms	14 ms
25,000	6.87 MB	1.91 MB	72.21%	16 ms	18 ms
100,000	27.47 MB	7.63 MB	72.22%	52 ms	65 ms

Memory grows linearly in both designs, but the flyweight slope is about 28% of the naive slope, and the shared pool stays at a handful of profiles (a function of the number of hardware models, not of the number of poles). The saving ratio therefore holds as the fleet grows, and the absolute saving grows with it. Creation time is similar and sometimes slightly higher for the flyweight, because every pole performs a cache lookup; the trade is a small CPU cost for a large memory saving.

Empirical check. Measuring live heap after `System.gc()` gave 4.80 MB for the naive fleet and 2.63 MB for the flyweight fleet, a 45.1% reduction. This is lower than the 72.2% model because real measurements include JVM and container overhead that the layout model excludes, so I report both figures and treat the 72.2% as an analytical estimate, not a profiler result.

Garbage collection. The shared profiles are strongly referenced by the static cache, so they stay reachable for the application's lifetime. The benefit is that far fewer objects are allocated, which lowers young-generation pressure and collection frequency. It does not eliminate garbage collection, and promotion between generations still depends on JVM tuning (Oracle, 2024).

Limits. Flyweight adds one level of indirection and requires profiles to stay immutable. If every light had a unique configuration the benefit would disappear, and a distributed deployment would need the pool synchronised across nodes.

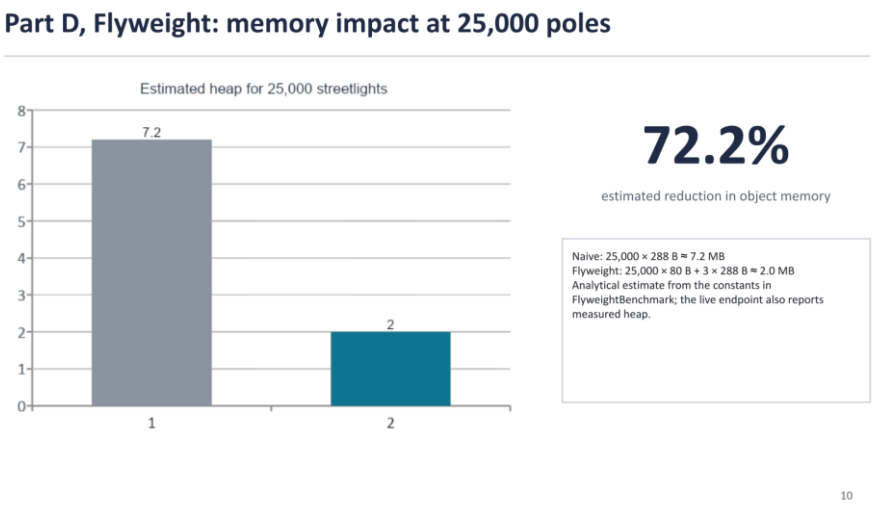


Figure 5.2: Estimated heap for 25,000 streetlights

6. Part E: Interpreting City Policies Defined in DSL

> *Operational rules such as noise limits and parking hours are written in a mini-language that the system must parse and evaluate dynamically.*

E.1 Identification and Justification (3 marks)

Selected pattern: Interpreter (behavioral; Gamma et al., 1994). The pattern represents a small language as a class hierarchy in which each grammar rule becomes a class, and a sentence is an abstract syntax tree (AST) of those objects. Rules look like `((NOISE_LEVEL > 75.0) AND (CURRENT_HOUR >= 22.0)) OR (AIR_QUALITY_AQI > 150.0)`. Interpreter fits because the language is small and declarative, and because each production maps directly to a class, so extending the grammar is straightforward. It also avoids general-purpose engines such as JavaScript `eval()` or Groovy, which expose the server to arbitrary code execution. Evaluation here is confined to the grammar's nodes.

E.2 UML Class Diagram and Mechanism (4 marks)

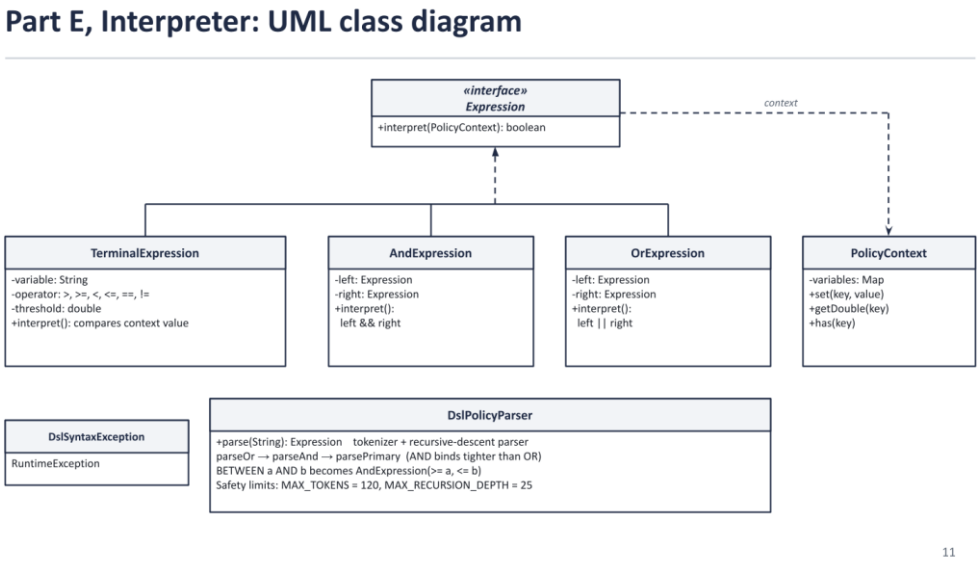


Figure 6.1: UML class diagram, Interpreter pattern (Part E)

The participants are: **Context** `PolicyContext` (holds live variables such as NOISE_LEVEL, case-insensitively); **AbstractExpression** `Expression` (`boolean interpret(PolicyContext)`); **TerminalExpression** (compares one variable with a numeric threshold using >, >=, <, <=, ==, !=, with an epsilon of 0.00001 for equality); and **non-terminals** `AndExpression` and `OrExpression` (short-circuit logic over two sub-expressions). `DslPolicyParser` is a recursive-descent parser that builds the tree from tokens and handles nested parentheses. `VAR BETWEEN a AND b` is compiled into an AND of two terminals (`VAR >= a` and `VAR <= b`). The parser enforces `MAX_TOKENS = 120` and `MAX_RECURSION_DEPTH = 25`, and malformed input raises `DslSyntaxException`. As a worked example, with NOISE_LEVEL = 78.5 and CURRENT_HOUR = 23, the rule `(NOISE_LEVEL > 75.0) AND (CURRENT_HOUR >= 22.0)` becomes an `AndExpression` over two terminals; both return true, so the policy is violated. Strictly, GoF leaves parsing outside the pattern; the Expression hierarchy is the pattern, and the parser is a supporting component.

E.3 Critical Assessment: Rule Updates and Extensibility (8 marks)

Rule updates. Policies are stored as DSL text in MySQL. When an administrator edits a rule through `POST /api/policies`, the next call to `POST /api/policies/evaluate` parses the new text. The parser and AST are stateless and built per request, so there is no shared mutable state between HTTP threads, no recompilation and no server restart. This suits ordinances that change often.

Performance. Over 2,000 iterations after JIT warm-up on OpenJDK 17, evaluating a pre-parsed tree took about 0.07 µs, parsing alone about 1.99 µs, and parse plus evaluation about 2.11 µs. This is fast enough for high-rate sensor ingestion even when rules are re-parsed on every request, and caching parsed trees is an easy further optimisation.

Extensibility. New operators are new classes that implement `Expression`, for example `NotExpression`, an `InListExpression` for zones, or a geospatial radius test, with no change to existing nodes or clients. This supports the Open/Closed Principle. Tests 28 to 35 cover relational operators, BETWEEN, non-numeric values, AND, OR, nesting, missing variables and five malformed-syntax cases.

Security. The DSL is non-Turing-complete, with no reflection, file access or system calls, so injected text cannot run code. This reduces the attack surface but does not remove it. Defence in depth is still needed: tokenisation rejects unknown symbols, the token and depth limits guard against resource-exhaustion payloads, and endpoint authorisation plus rate limiting should protect rule submission.

Limitations.

- **Class proliferation.** Every new grammar rule needs a class. For four node types this is clean, but adding IF-THEN-ELSE, loops or functions would make the hierarchy and its tests unmanageable.
- **Hand-written parser risk.** Operator precedence, associativity and error recovery become bug-prone as the grammar grows; formal tools such as ANTLR or JavaCC handle them better.
- **Fail-silent variables.** An unregistered variable evaluates to false. This avoids null pointer errors, but a mistyped variable name in a violation rule would quietly report "no violation". A stricter design would reject unknown variable names at parse time.
- **Wrong tool for complex logic.** If rules ever need arbitrary computation, a parser generator or a sandboxed scripting engine (for example GraalVM) is the better choice.

For SmartCity360, where policies are declarative Boolean thresholds, the Interpreter is a suitable, dependency-free and safe choice. The pattern would stop being appropriate only if the language became a general programming language.

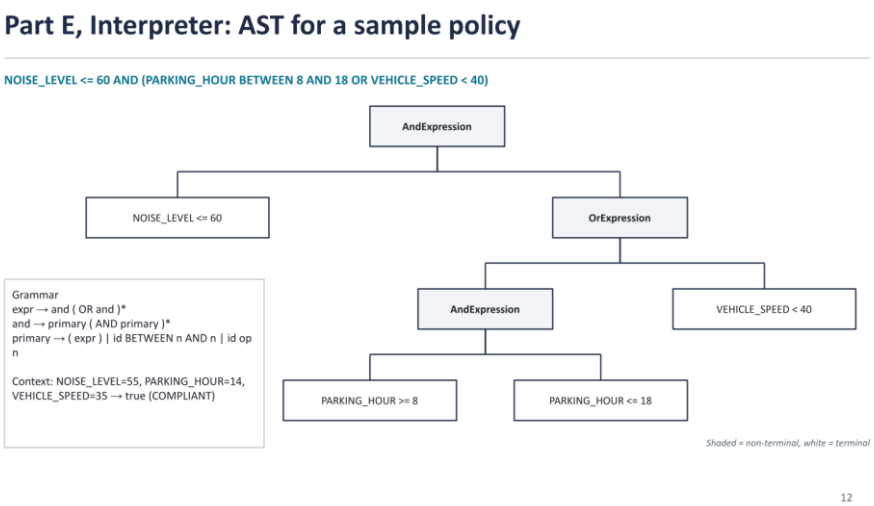


Figure 6.2: AST for a sample policy

7. Part F: Final Project Demonstration, VIVA and Documentation

7.1 System Architecture

SmartCity360 is a four-tier system following enterprise layering principles (Fowler, 2002):

- **Presentation tier:** a Next.js web application with management panels for streams, zones, complaints, streetlights, policies and the test centre.
- **API tier:** Java 17 servlets (`/api/auth`, `/api/streams`, `/api/zones`, `/api/complaints`, `/api/streetlights`, `/api/policies`, `/api/dashboard`, `/api/test-center`) with CORS and token-authentication filters and Gson JSON serialisation.
- **Pattern domain core:** the five pattern packages under `com.smartcity.pattern.*`, free of framework dependencies.
- **Persistence tier:** MySQL 8 with nine tables, accessed through DAO classes, a custom connection pool and parameterised prepared statements that prevent SQL injection.

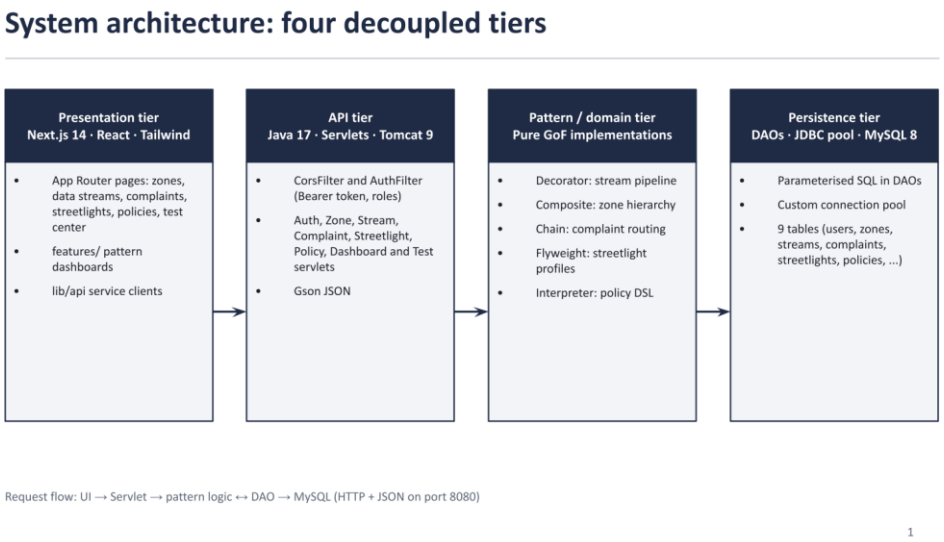


Figure 7.1: System architecture

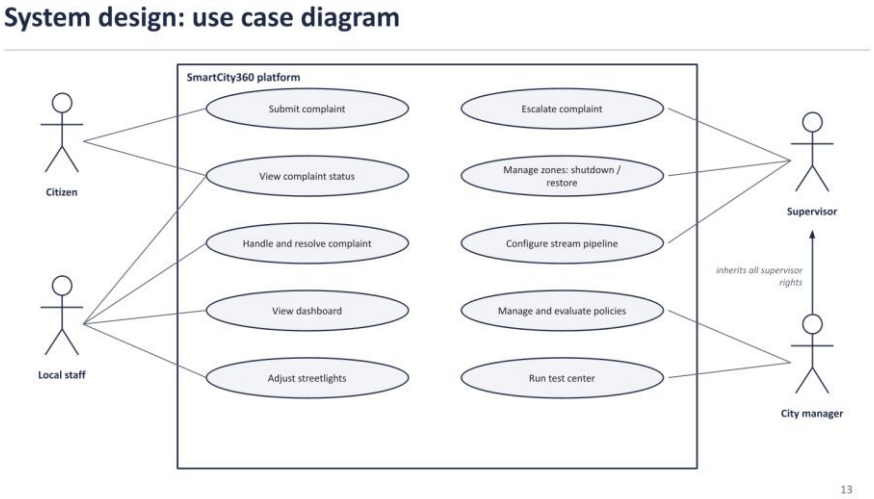


Figure 7.2: Use case diagram

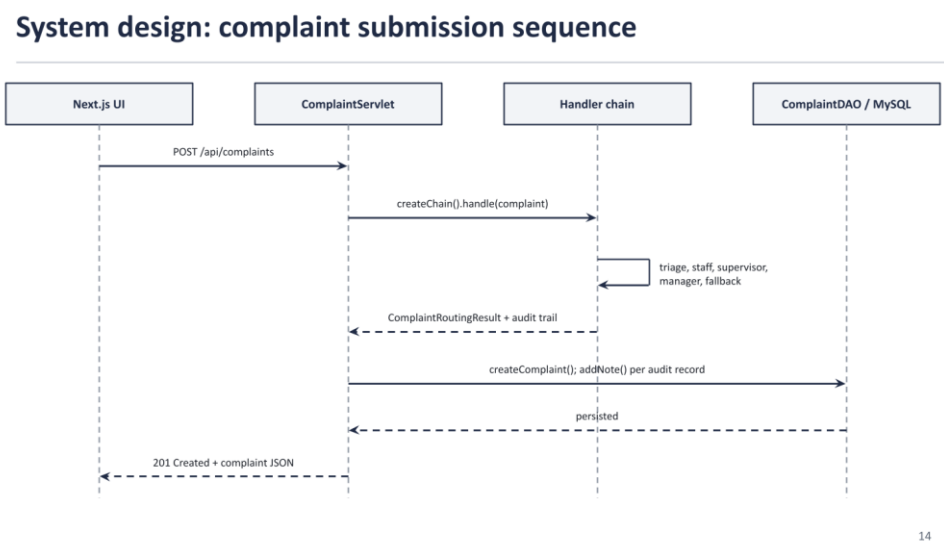


Figure 7.3: Complaint submission sequence diagram

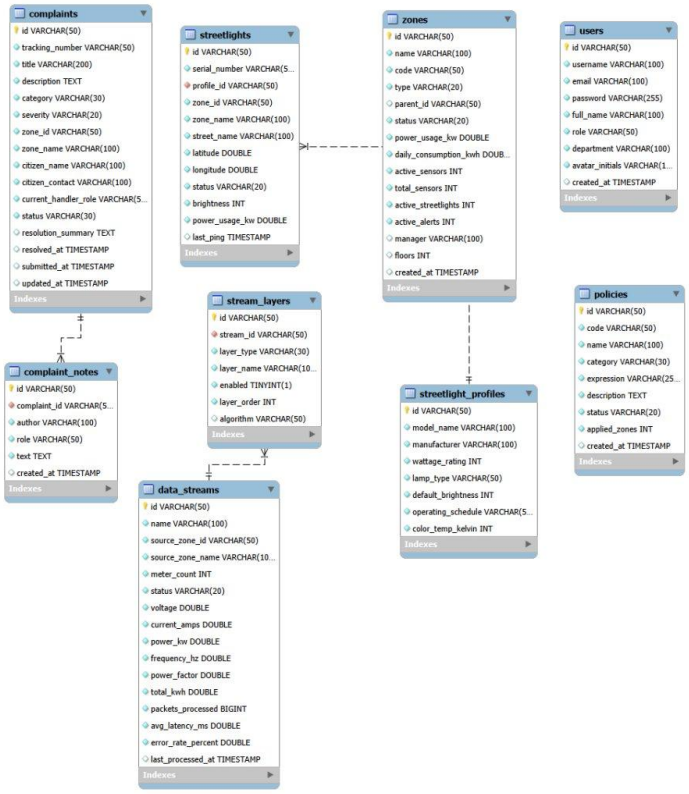


Figure 7.4: Entity-relationship diagram

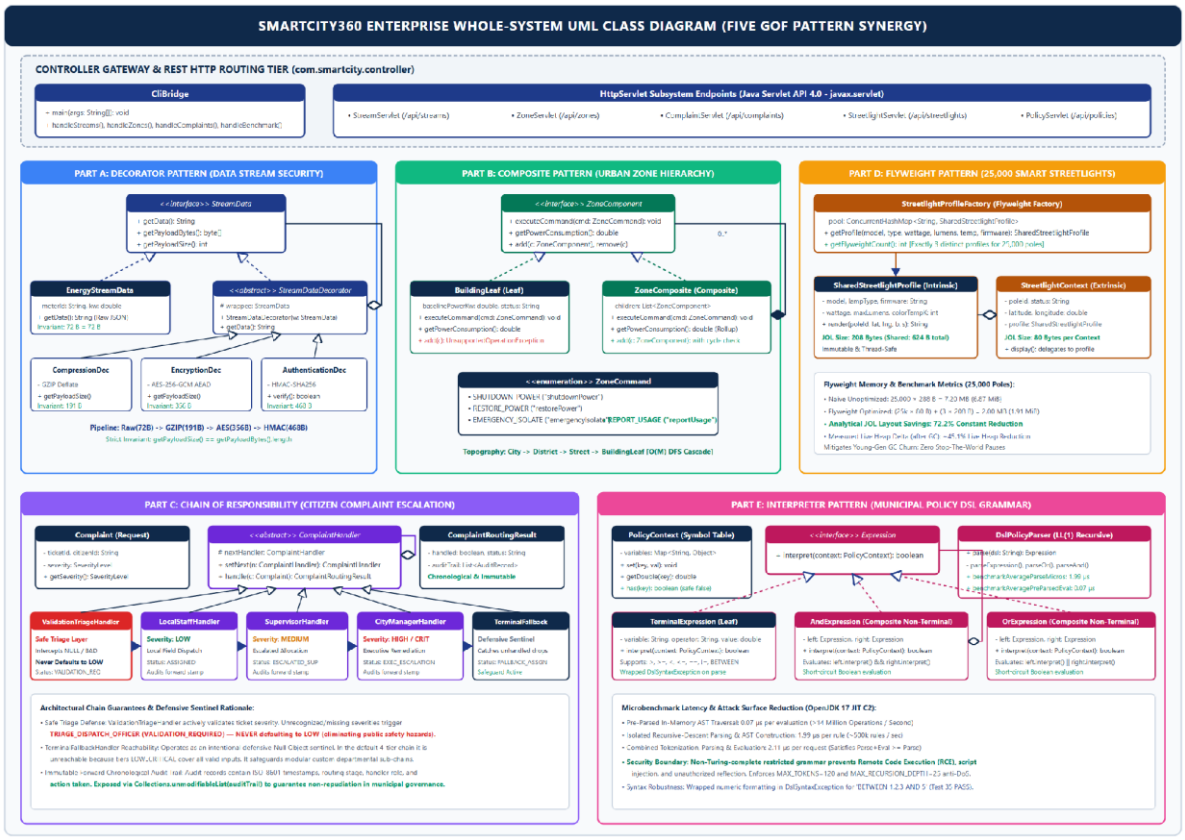


Figure 7.5: Whole-system UML class diagram

7.2 Key Architectural Choices and Anti-Pattern Mitigation

Anti-pattern	How SmartCity360 avoids it
Spaghetti code, Long method	Procedural branching is replaced by small single-purpose pattern classes (decorators, handlers, expression nodes).
God object	Responsibilities are spread across decorators, handlers and composite nodes rather than one manager class.
Copy-paste programming	Shared behaviour lives in base types: `StreamDataDecorator`, `ComplaintHandler`, `ZoneComponent`.
Tight coupling	Clients depend on interfaces (`StreamData`, `ZoneComponent`, `Expression`), not concrete classes.
Primitive obsession, Magic numbers	Enums and value objects (`ZoneCommand`, `ComplaintSeverity`, `ComplaintRoutingResult`) and named constants replace raw strings and literals.
Swallowed exceptions	Errors propagate as typed exceptions (`DslSyntaxException`, `AEADBadTagException`); fallback paths are recorded.

7.3 Automated Test Results

`ComprehensiveTestSuite` is a zero-dependency test harness (no JUnit), so it runs on any clean JDK 17 machine and feeds the application's Test Center page. All 35 tests passed (100%).

Pattern	Tests	Passed	Main behaviours verified
Decorator	6	6	Pipeline order, decryption, HMAC rejection, corrupted data, invalid key

Pattern	Tests	Passed	Main behaviours verified
Composite	8	8	Cascade, restore, aggregation, leaf safety, cycle rejection
Chain of Responsibility	8	8	Severity routing, safe triage, fallback, audit trail
Flyweight	5	5	Instance sharing, thread safety, 72.2% layout saving
Interpreter	8	8	AST, BETWEEN, nesting, missing variables, malformed syntax
Total	**35**	**35**	**100% pass rate**

7.4 VIVA Preparation and Demonstration Video Walkthrough

For the VIVA presentation defense, each pattern selection is justified against architectural alternatives (Sections 2 through 6), explaining measured trade-offs and code implementation details:

Likely question	Short answer
Why compress before encrypting?	Ciphertext is statistically random and cannot be compressed, so compression must come first.
Why can a Composite leaf throw on `add()`?	The interface favours transparency; the leaf guards invalid structural operations at runtime.
When is the fallback handler reached?	Only in reconfigured chains that omit a tier; the standard chain covers all severities.
Is the 72.2% saving measured?	It is an analytical layout estimate; live heap measurement gave 45.1%.
Why not use a scripting engine for policies?	Rules are simple Boolean thresholds, and a restricted DSL has a much smaller attack surface.

The 10 to 15 minute demonstration video covers, in sequence: the architectural overview and technology stack (2 minutes); live demonstrations of all five Gang of Four patterns through the web interface and REST API (2 minutes per pattern); execution of the automated test harness with 35 of 35 verified passes (1 minute); and a closing evaluation of design trade-offs and production extensibility.

7.5 Submission Deliverables (Part F.5)

Requirement	Student Portal section	Artifact
5.a Project documentation	Project Document (PDF)	This report, exported as a PDF
5.b Source code	Project File (ZIP)	Complete project source archive
5.c Demonstration video	Demonstration Video (MP4)	10 to 15 minute recorded demonstration

8. Conclusion and References

8.1 Conclusion

Each pattern was selected for a distinct problem: Decorator for layered stream security, Composite for the zone hierarchy, Chain of Responsibility for traceable escalation, Flyweight for memory efficiency across 25,000 lights, and Interpreter for safe runtime policy evaluation. Evaluating each pattern also exposed its costs, such as ordering rules for decorators, latency and debugging effort in chains, and class growth in interpreters. The implementation is verified by 35 passing tests.

Future improvements. (1) Migrate the policy language to ANTLR or a sandboxed engine if it grows beyond Boolean thresholds. (2) Synchronise the flyweight pool and zone state across cluster nodes with a distributed cache. (3) Use asymmetric signatures such as ECDSA where non-repudiation is required.

8.2 References

Coward, D. (2017). Java Servlet Specification, Version 4.0 (JSR 369). Oracle and Eclipse Foundation.

Dworkin, M. (2007). Recommendation for Block Cipher Modes of Operation: GCM and GMAC. NIST SP 800-38D.

Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Goetz, B. et al. (2006). Java Concurrency in Practice. Addison-Wesley.

Java Institute for Advanced Technology. (2026). Object Oriented Design Patterns II Lecture Notes.

Krawczyk, H., Bellare, M. and Canetti, R. (1997). HMAC: Keyed-Hashing for Message Authentication. RFC 2104. IETF.

Martin, R. C. (2008). Clean Code. Prentice Hall.

Oracle. (2024). Java SE 17 Developer Documentation and HotSpot Memory Management Guide.

Appendix A: Source Code Screenshot Catalog

Working source files are grouped by pattern below. The complete compilable project is in the submitted ZIP.

A.1 Decorator (com.smartcity.pattern.decorator)



```java
package com.smartcity.pattern.decorator;

public interface StreamData {

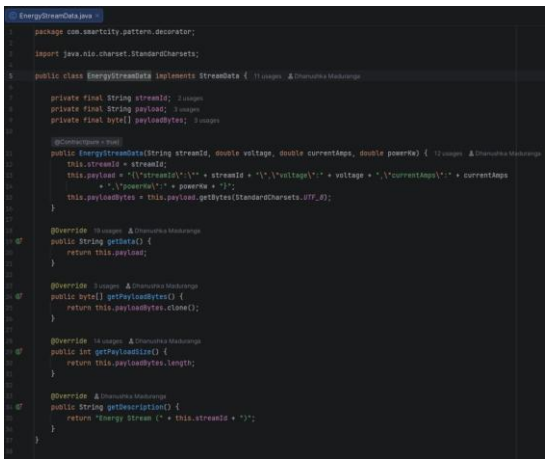
    String getData();

    byte[] getPayloadBytes();

    int getPayloadSize();

    String getDescription();
}
```

StreamData.java (component interface)



```java
package com.smartcity.pattern.decorator;

import java.nio.charset.StandardCharsets;

public class EnergyStreamData implements StreamData {

    private final String streamId;
    private final String payload;
    private final byte[] payloadBytes;

    public EnergyStreamData(String streamId, double voltage, double currentAmps, double powerKw) {
        this.streamId = streamId;
        this.payload = "{\"streamId\":\"" + streamId + "\",\"voltage\":" + voltage + ",\"currentAmps\":" + currentAmps
                + ",\"powerKw\":" + powerKw + "}";
        this.payloadBytes = this.payload.getBytes(StandardCharsets.UTF_8);
    }

    @Override
    public String getData() {
        return this.payload;
    }

    @Override
    public byte[] getPayloadBytes() {
        return this.payloadBytes.clone();
    }

    @Override
    public int getPayloadSize() {
        return this.payloadBytes.length;
    }

    @Override
    public String getDescription() {
        return "Energy Stream (" + this.streamId + ")";
    }
}
```

EnergyStreamData.java (concrete component)



```java
package com.smartcity.pattern.decorator;

public abstract class StreamDataDecorator implements StreamData {

    protected final StreamData wrappee;

    public StreamDataDecorator(StreamData wrappee) {
        if (wrappee == null) {
            throw new IllegalArgumentException("Wrappee component cannot be null");
        }
        this.wrappee = wrappee;
    }

    public StreamData getWrappee() {
        return this.wrappee;
    }

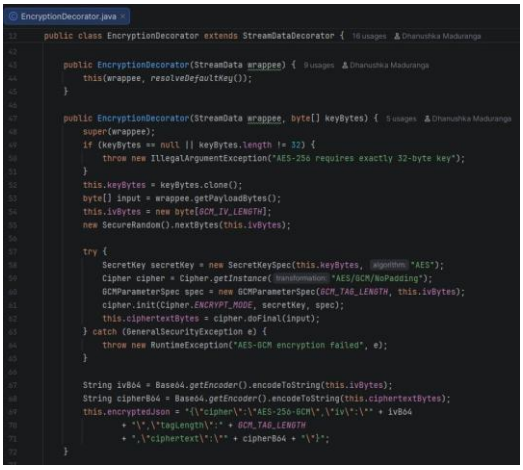
    @Override
    public String getData() {
        return this.wrappee.getData();
    }

    @Override
    public byte[] getPayloadBytes() {
        return this.wrappee.getPayloadBytes();
    }

    @Override
    public int getPayloadSize() {
        return this.wrappee.getPayloadSize();
    }

    @Override
    public String getDescription() {
        return this.wrappee.getDescription();
    }
}
```

StreamDataDecorator.java (abstract decorator)



```java
public class EncryptionDecorator extends StreamDataDecorator {

    public EncryptionDecorator(StreamData wrappee) {
        this(wrappee, resolveDefaultKey());
    }

    public EncryptionDecorator(StreamData wrappee, byte[] keyBytes) {
        super(wrappee);
        if (keyBytes == null || keyBytes.length != 32) {
            throw new IllegalArgumentException("AES-256 requires exactly 32-byte key");
        }
        this.keyBytes = keyBytes.clone();
        byte[] input = wrappee.getPayloadBytes();
        this.ivBytes = new byte[GCM_IV_LENGTH];
        new SecureRandom().nextBytes(this.ivBytes);

        try {
            SecretKey secretKey = new SecretKeySpec(this.keyBytes, "AES");
            Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
            GCMParameterSpec spec = new GCMParameterSpec(GCM_TAG_LENGTH, this.ivBytes);
            cipher.init(Cipher.ENCRYPT_MODE, secretKey, spec);
            this.ciphertextBytes = cipher.doFinal(input);
        } catch (GeneralSecurityException e) {
            throw new RuntimeException("AES-GCM encryption failed", e);
        }

        String ivB64 = Base64.getEncoder().encodeToString(this.ivBytes);
        String cipherB64 = Base64.getEncoder().encodeToString(this.ciphertextBytes);
        this.encryptedJson = "{\"cipher\":\"AES-256-GCM\",\"iv\":\"" + ivB64
                + "\",\"tagLength\":" + GCM_TAG_LENGTH
                + ",\"ciphertext\":\"" + cipherB64 + "\"}";
    }
```

EncryptionDecorator.java (AES-256-GCM)



```java
package com.smartcity.pattern.decorator;

import java.io.ByteArrayInputStream;
import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.util.Base64;
import java.util.zip.GZIPInputStream;
import java.util.zip.GZIPOutputStream;

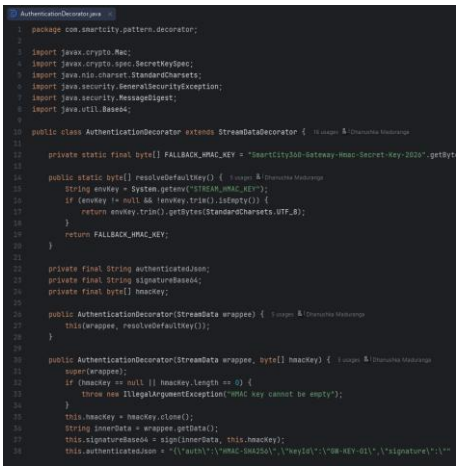
public class CompressionDecorator extends StreamDataDecorator {

    private final String compressedJson;
    private final byte[] compressedBytes;
    private final int originalSize;
    private final int compressedSize;

    public CompressionDecorator(StreamData wrappee) {
        super(wrappee);
        byte[] input = wrappee.getPayloadBytes();
        this.originalSize = input.length;
        this.compressedBytes = compressBytes(input);
        this.compressedSize = this.compressedBytes.length;
        String b64 = Base64.getEncoder().encodeToString(this.compressedBytes);
        this.compressedJson = "{\"format\":\"GZIP\",\"originalBytes\":" + this.originalSize
                + ",\"compressedBytes\":" + this.compressedSize
                + ",\"data\":\"" + b64 + "\"}";
    }

    private static byte[] compressBytes(byte[] data) {
        try {
            ByteArrayOutputStream baos = new ByteArrayOutputStream();
            try (GZIPOutputStream gzip = new GZIPOutputStream(baos)) {
                gzip.write(data);
                gzip.finish();
            }
            return baos.toByteArray();
        } catch (IOException e) {
```

CompressionDecorator.java (GZIP)



```java
package com.smartcity.pattern.decorator;

import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import java.nio.charset.StandardCharsets;
import java.security.GeneralSecurityException;
import java.security.MessageDigest;
import java.util.Base64;

public class AuthenticationDecorator extends StreamDataDecorator {

    private static final byte[] FALLBACK_HMAC_KEY = "SmartCity360-Gateway-Hmac-Secret-Key-2026".getByt

    public static byte[] resolveDefaultKey() {
        String envKey = System.getenv("STREAM_HMAC_KEY");
        if (envKey != null && !envKey.trim().isEmpty()) {
            return envKey.trim().getBytes(StandardCharsets.UTF_8);
        }
        return FALLBACK_HMAC_KEY;
    }

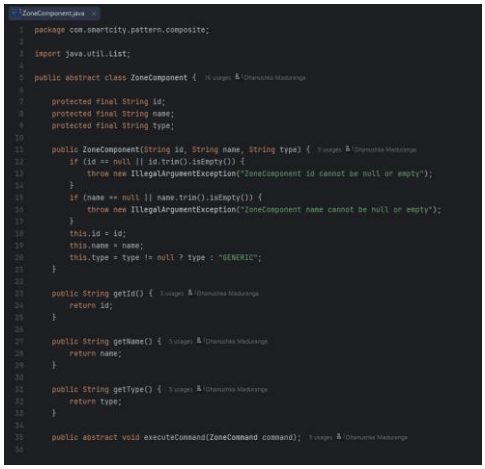
    private final String authenticatedJson;
    private final String signatureBase64;
    private final byte[] hmacKey;

    public AuthenticationDecorator(StreamData wrappee) {
        this(wrappee, resolveDefaultKey());
    }

    public AuthenticationDecorator(StreamData wrappee, byte[] hmacKey) {
        super(wrappee);
        if (hmacKey == null || hmacKey.length == 0) {
            throw new IllegalArgumentException("HMAC key cannot be empty");
        }
        this.hmacKey = hmacKey.clone();
        String innerData = wrappee.getData();
        this.signatureBase64 = sign(innerData, this.hmacKey);
        this.authenticatedJson = "{\"auth\":\"HMAC-SHA256\",\"keyId\":\"SM-KEY-01\",\"signature\":\""
```

AuthenticationDecorator.java (HMAC-SHA256)

A.2 Composite (com.smartcity.pattern.composite)



ZoneComponent.java



BuildingLeaf.java

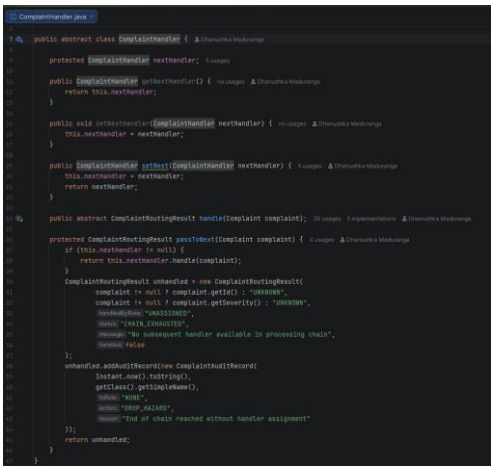


ZoneComposite.java

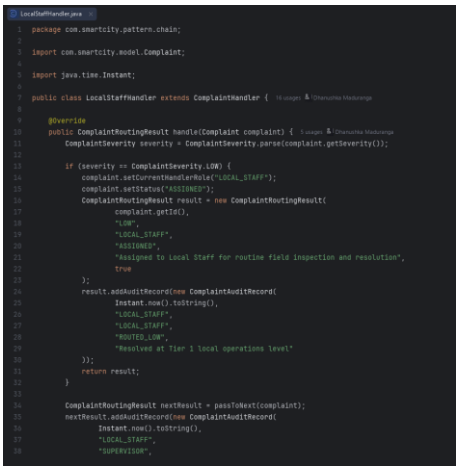


ZoneCommand.java

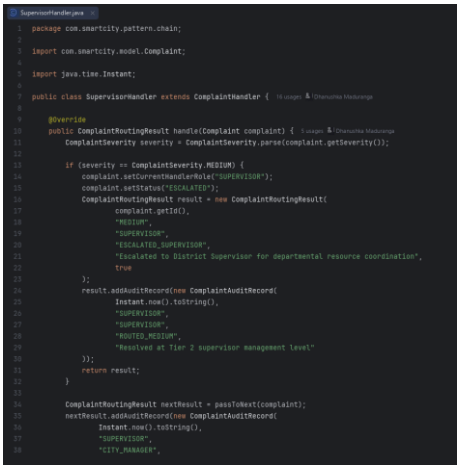
A.3 Chain of Responsibility (com.smartcity.pattern.chain)



ComplaintHandler.java



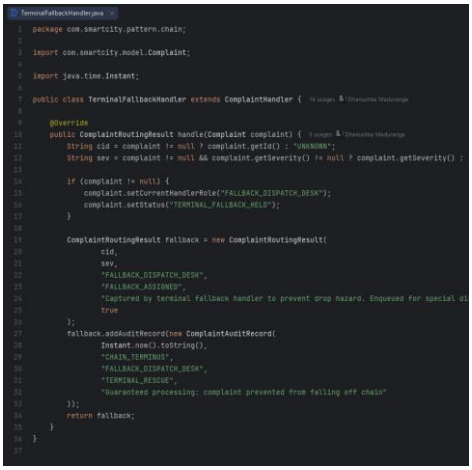
LocalStaffHandler.java



SupervisorHandler.java



CityManagerHandler.java

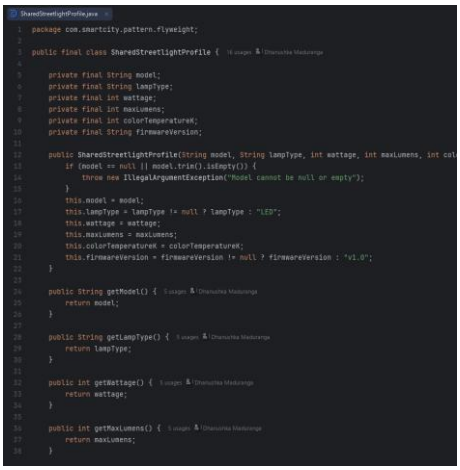


TerminalFallbackHandler.java

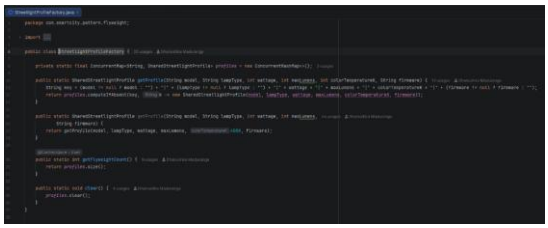


ComplaintChainFactory.java

A.4 Flyweight (com.smartcity.pattern.flyweight)



SharedStreetlightProfile.java



StreetlightProfileFactory.java



StreetlightContext.java

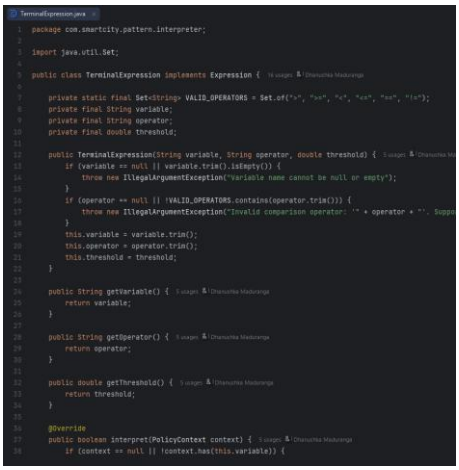


FlyweightBenchmark.java

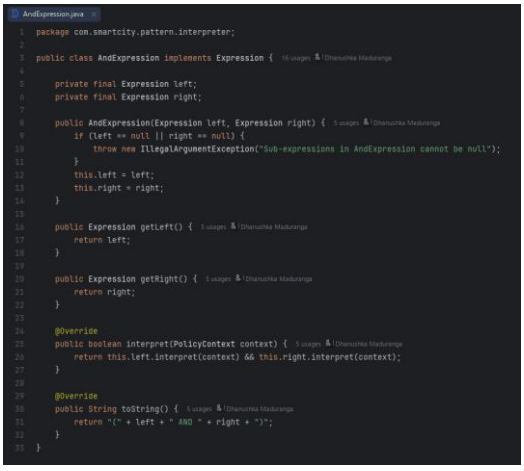
A.5 Interpreter (com.smartcity.pattern.interpreter)



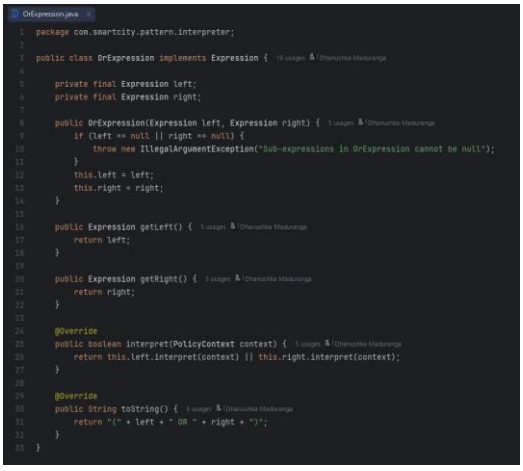
Expression.java



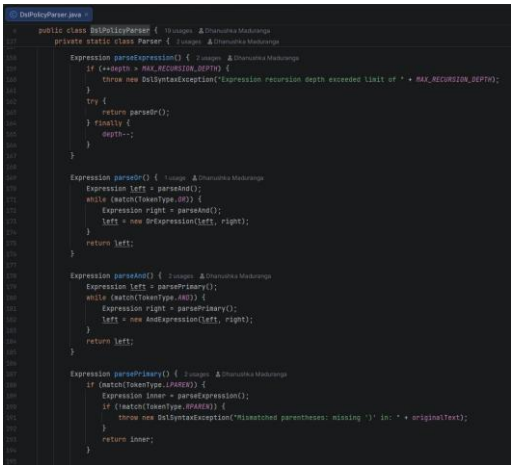
TerminalExpression.java



AndExpression.java



OrExpression.java



DslPolicyParser.java



PolicyContext.java